

Tunisia Heritage Quest Android Application Report

Rana Ben Fraj

1 Introduction

Tunisia Heritage Quest is an Android application developed using Kotlin and Jetpack Compose. The purpose of this project is to create an interactive quiz game that allows users to identify Tunisian historical monuments from images and answer related questions.

The application is designed for students, tourists, and anyone interested in learning about Tunisian heritage in a simple and engaging way.

2 Project Overview

The application is based on a quiz system with the following features:

- Image-based multiple-choice questions
- Three difficulty levels (easy, medium, hard)
- Score system (+10 points per correct answer)
- Progress tracking (mastery percentage)
- Streak tracking (number of completed games)
- Final result screen with performance evaluation

Currently, only one category (Roman Heritage) is implemented.

3 System Architecture

The application follows a modern Android architecture using:

- **Jetpack Compose:** Used to design all user interface screens
- **Navigation Component:** Handles transitions between screens (Splash, Menu, Category, Quiz, Results)

- **ViewModel:** Manages application state such as score and current question
- **State Management:** Implemented using `mutableStateOf` to update UI dynamically

Data Model:

```
data class QuizQuestion(
    val imageRes: Int,
    val question: String,
    val options: List<String>,
    val correctAnswer: String,
    val difficulty: String,
    val explanation: String
)
```

4 Application Features

The main features developed include:

- Splash screen with automatic navigation
- Main menu with statistics (sites, mastery, streak)
- Category selection screen
- Difficulty selection screen
- Quiz screen with image and multiple answers
- Answer validation with feedback and explanation
- Result screen with score and percentage

5 Use of Artificial Intelligence

Artificial Intelligence (AI), specifically GitHub Copilot, was used throughout the development process as a coding assistant.

5.1 UI Development

AI was used to generate Jetpack Compose code for all screens:

- Layout structures (Column, Row, Card, Button)
- Screen organization and spacing
- Material Design styling

This helped speed up UI creation while maintaining a clean design.

5.2 Question Generation

AI played an important role in generating quiz questions:

- Easy level: identifying monuments from images
- Medium level: questions about historical details
- Hard level: deeper facts such as dates and historical context

The questions were varied and avoided repetition, making the quiz more engaging.

5.3 State Management

AI helped implement state handling using ViewModel and mutable states:

- Tracking current question index
- Managing selected answers
- Updating UI dynamically

5.4 Debugging and Error Fixing

AI was used to:

- Fix compilation errors
- Resolve navigation issues
- Prevent crashes (e.g., index out of bounds, null values)

5.5 Testing

AI assisted in generating testing code for:

- Unit testing (ViewModel)
- UI testing (Compose)
- Navigation testing